



A Course End Project Report on
Student Performance prediction-Basic

A9513 – PYTHON PROGRAMMING LAB

Submitted by:

Name: NADIMINTI NANDINI

RollNo: 25881A0599

Name: C.SRUTHI

RollNo: 26885A0513

Course Facilitator

Mr /Ms S Madhavi

Assistant Professor

Department of Computer Science and Engineering Vardhaman College of
Engineering, Hyderabad

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled “**Student performance prediction-basic**” is carried out by **NADIMINTI.NANDINI** and **C.SRUTHI** with Roll Numbers **25881A0599** and **26885A0513** towards **A9513 – Python Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology in Computer Science and Engineering** during the Academic year 2026-27.

Signature of the Course Faculty Mr /Ms
Madhavi
Assistant Professor, CSE

Signature of the HOD
Dr.Vijayalatha,HOD & dean(student
monitoring), CSE,Vardhaman college of
engineering

Contents

	Introduction.....	2
1	Literature Review	3
2	Objectives 3	
3	Methodology 3	
3.1	Architectural Diagram	3
3.2	Hardware and Software Requirements.....	3
3.3	Working Principle 4	
3.4	Software Implementation.....	4
4	Results and Discussion	4
5	Mapping POs and SDGs.....	5
5.1	Program Outcomes (POs) Mapping.....	5
5.2	Sustainable Development Goals (SDGs) Mapping.....	5
6	Conclusion 5	
	Bibliography	6

Abstract

Abstract

The **Student Performance Prediction** project is a Python-based application developed to analyze, monitor, and predict the academic performance of students using their marks and attendance data. In educational institutions, manually monitoring the performance of a large number of students can be time-consuming and difficult. This project provides an efficient system for managing student information and analyzing academic performance. The system considers student details such as Roll Number, Name, subject-wise marks, attendance, and final marks in subjects including Python, Java, DBMS, Operating Systems, and Discrete Mathematics.

The system uses **Pandas and NumPy** for data processing and analysis, **Matplotlib** for graphical visualization, and **Scikit-learn** for implementing machine learning-based performance prediction. It provides features such as **Studies Performance, Attendance Performance, Overall Performance, Prediction Performance, and Class Comparison**. The GUI-based application allows users to easily view and analyze student performance without directly interacting with complex code. Database operations are also used to store and retrieve student-related information efficiently.

The main objective of the project is to help faculty understand student performance and identify students who may require additional academic support. By analyzing marks, attendance, and overall performance, the system provides useful insights for academic monitoring and decision-making. The project demonstrates the practical application of **Python programming, data analysis, machine learning, data visualization, GUI development, and database concepts** in the education domain and provides a simple and effective approach to student performance evaluation and prediction.

Keywords: Student Performance Prediction, Machine Learning, Random Forest, Educational Data Mining, Regression

1.INTRODUCTION

Student academic performance is one of the important factors used to evaluate the learning progress and development of students in educational institutions. Faculty members regularly monitor students through their marks, attendance, assignments, internal examinations, and final examinations. However, when the number of students and subjects increases, manually collecting, processing, and comparing this information becomes time-consuming and difficult. A computerized system can make this process easier by organizing student information and providing meaningful performance analysis.

The Student Performance Prediction project is developed as a Python-based application to analyze and predict student academic performance using marks and attendance data. The system maintains important information such as student Roll Number, Name, subject-wise marks, subject-wise attendance, and final marks. The project considers multiple subjects, including Python, Java, DBMS, Operating Systems, and Discrete Mathematics, to provide a broader view of a student's academic performance. By combining different academic factors, the system provides a more structured approach to performance monitoring.

Python is used as the primary programming language because it provides powerful libraries and tools for data processing, visualization, and machine learning. Pandas and NumPy are used for handling and processing student datasets, performing calculations, and preparing data for analysis. Matplotlib is used to represent student performance through graphs and charts, which makes the results easier to understand. Scikit-learn is used to implement the machine learning model for predicting student performance based on the available academic and attendance information.

The application provides different performance analysis features, including Studies Performance, Attendance Performance, Overall Performance, Prediction Performance, and Class Comparison. Studies Performance focuses on academic marks, while Attendance Performance helps analyze attendance levels. Overall Performance provides a combined view of the student's academic status. Prediction Performance uses the trained machine learning model to provide an estimated performance result. Class Comparison allows an individual student's performance to be compared with the overall class performance, helping faculty understand the student's position within the class.

The project also incorporates Graphical User Interface (GUI) and database concepts to make the application simple and convenient to use. The GUI allows users to access different functions through buttons and view the required results without directly executing individual Python commands. Database operations help in storing and retrieving student-related records efficiently. This integration of programming, data analysis, machine learning, visualization, GUI, and database concepts makes the project a practical application of the concepts learned in the Python Programming Lab.

The main purpose of this project is to support faculty in monitoring student performance and identifying students who may require additional academic attention. The system can provide useful insights by analyzing marks and attendance and presenting the results in an understandable form. It can also help faculty make better academic decisions based on available data rather than relying only on manual observation.

Overall, the Student Performance Prediction project demonstrates how Python and machine learning can be applied to solve a real-world problem in the education domain. The project provides an organized and efficient method for analyzing student performance, comparing academic results, and predicting performance. It also provides practical experience in Python programming, data preprocessing, data visualization, machine learning, GUI development, and database integration, making it a useful academic project with real-world relevance.

2.LITERATURE REVIEW

Predicting student performance has been studied widely under Educational Data Mining (EDM) and Learning Analytics. Early work focused on using student records to understand learning behavior. Baker and Yacef (2009) and Romero and Ventura (2010) reviewed the field and identified performance prediction as one of its most common applications. They noted that decision trees, regression, and neural networks were the most frequently used techniques.

Cortez and Silva (2008) studied secondary school students in Portugal using Decision Trees, Random Forest, Neural Networks, and Support Vector Machines. Their dataset, now available in the UCI repository, is the one used in this project. They found that prior period grades (G1 and G2) were the strongest predictors of the final grade, and that accuracy dropped when those grades were removed. Kotsiantiset al. (2004) applied machine learning to distance learning students and showed that early prediction of weak students is possible, so timely help can be given.

Later studies compared algorithms on different datasets. Most found that ensemble methods such as Random Forest usually outperform single models like Linear Regression or a single Decision Tree. Researchers also found that non-academic factors such as absences, study time, past failures, parental education, and family support improve predictions. Simpler models, however, remain easier to interpret, which matters when teachers need to understand why a student is flagged.

Research gap: Many existing studies use complex models or large datasets that are hard for beginners to apply. This project fills that gap with a basic, interpretable system using standard algorithms on a well-known dataset, with the goal of giving a clear and simple early-warning tool.

Attendance is another important factor frequently considered in student performance analysis. Regular attendance can provide an indication of student participation in academic activities. Several educational analysis systems combine attendance information with examination results to obtain a more complete understanding of student performance. In the proposed project, subject-wise attendance is considered along with academic marks so that performance can be analyzed from both academic and participation perspectives. Data visualization is also widely used in educational data analysis because numerical datasets can be difficult to understand directly. Graphs and charts can provide a clear representation of subject-wise marks, attendance levels, and overall performance. Visualization allows faculty members to identify differences between students, observe performance patterns, and compare individual results with class-level results. In this project, **Matplotlib** is used to generate graphical representations that make the analysis easier to interpret.

Python has become a widely used programming language for data analysis and machine learning because of its extensive collection of libraries. **Pandas** provides efficient methods for reading, manipulating, filtering, and analyzing datasets, while **NumPy** supports numerical calculations and data processing. **Matplotlib** provides visualization capabilities, and **Scikit-learn** provides machine learning algorithms and evaluation tools. The availability of these libraries makes Python suitable for developing an integrated student performance analysis and prediction system.

Existing student performance prediction approaches often focus mainly on predicting final grades or identifying students who are likely to fail. However, an effective academic monitoring system can provide additional analysis rather than only producing a prediction result. Therefore, the proposed project combines multiple functions, including **Studies Performance, Attendance Performance, Overall Performance, Prediction Performance, and Class Comparison**. This provides a broader view of student performance and makes the system more useful for academic monitoring.

The proposed system also incorporates a **Graphical User Interface (GUI)** and database operations. A GUI provides a simple way for faculty members to interact with the system and access different performance analysis features. Database integration allows student information to be stored and retrieved systematically instead of maintaining data only in separate files. The combination of these components provides a more organized and practical solution for student performance monitoring.

From the existing literature and approaches, it can be observed that student performance prediction can benefit from combining academic data, attendance information, data visualization, and machine learning techniques. However, many systems focus on a single prediction task or a limited set of performance factors. The proposed project attempts to provide a simple and integrated solution by combining **student data management, academic analysis, attendance analysis, visualization, machine learning prediction, and class comparison** within a single Python-based application.

3.Objectives

.To design and develop a machine learning model that predicts whether a student will pass or fail.

The primary objective of this project is to build a functional and efficient student performance prediction system using Python. The system will use commonly used classification algorithms such as Logistic Regression, Decision Tree, and Random Forest. The model learns from academic, personal, and social attributes of students, such as previous grades, weekly study time, attendance, past failures, and parental education, and converts the final grade into a binary Pass or Fail outcome. The aim is to mimic how a teacher judges a student's progress, but in a data-driven and consistent way.

. To perform data preprocessing, exploratory analysis, and visualization of student data.

Raw student data is rarely ready for modeling, so this objective focuses on cleaning and preparing it. It includes handling missing values, encoding categorical variables, scaling numerical features, and splitting the data into training and testing sets. Exploratory data analysis with charts such as histograms, correlation heatmaps, and bar plots will then show how factors like study time, absences, and past failures relate to passing or failing. This visual representation helps users understand which factors matter most before any model is trained.

.To evaluate and compare the models using performance metrics such as accuracy, precision, recall, and F1-score.

This objective involves measuring how well each algorithm predicts pass and fail outcomes, using accuracy, precision, recall, F1-score, and the confusion matrix. The project will compare the results of the different models to show how performance varies with the algorithm used, and will select the best-performing one. It will also identify the most influential features, such as previous grades and attendance. By presenting the results in tables and graphs, users can better understand the trade-offs between accuracy and reliability, and how the model can serve as an early-warning tool to help teachers support at-risk students in time.

4.Methodology

4.1Architectural Diagram

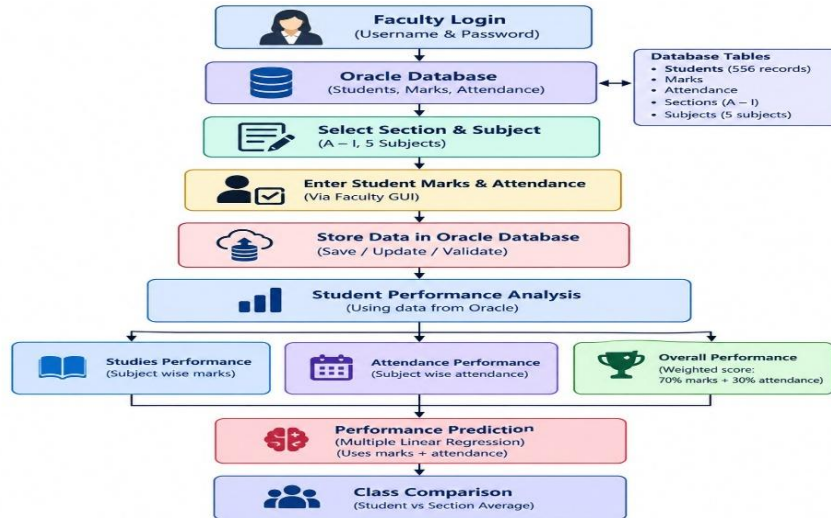


Figure 1: Architectural Diagram

The Data Collection and Preprocessing Module is the starting point of the system, where the student dataset is loaded and prepared for modeling. The dataset contains attributes such as previous grades, weekly study time, attendance, past failures, and parental education. This module handles missing values, encodes categorical variables, scales numerical features, and converts the final grade into a binary Pass or Fail target. The data is then split into training and testing sets, so the models can be trained on one part and tested fairly on unseen data.

The Machine Learning Engine forms the system's core, training the classification algorithms on the prepared data. It implements Logistic Regression, Decision Tree, and Random Forest, learns the patterns that separate passing students from failing ones, and predicts the outcome for new students. Because every model is trained on the same data, the comparison between them is fair and consistent.

The Evaluation and Visualization Module converts the model results into clear visual and tabular outputs. It calculates accuracy, precision, recall, and F1-score, and generates a confusion matrix for each model. It also produces charts such as correlation heatmaps and feature importance plots, which show the factors influencing student performance most. These outputs help users compare the algorithms and select the best-performing model.

Together, these components form a modular and extensible prediction framework. The design supports future upgrades such as additional algorithms, larger datasets, hyperparameter tuning, and a web-based interface where teachers can enter a student's details and instantly see a prediction, bridging theoretical understanding with practical early-warning use.

4.2 Hardware and Software Requirements

Hardware:

Intel/AMD Processor (Intel Core i3 or higher)

- Minimum 4GB RAM
- Minimum 500MB of free disk space
- Standard Display and Input Devices (keyboard and mouse)

Software:

- Operating System: Windows 10/11, Linux, or macOS
- Python 3.8 or higher
- Python Libraries: NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn
- Any IDE or notebook environment (Jupyter Notebook, Google Colab, VS Code, or PyCharm)
- Dataset: UCI Student Performance dataset (CSV format)

2.3 Working Principle

The working principle of this Student Performance Prediction system is based on replicating how a teacher judges a student's progress, but in a data-driven and consistent way. In real classrooms, teachers look at attributes such as previous grades, attendance, study habits, and past failures to decide which students may struggle. Similarly, this system accepts historical student records and feeds them into machine learning algorithms, where each algorithm applies its own strategy to learn patterns that separate passing students from failing ones. By learning these patterns step by step, the system demonstrates how different factors and different algorithms affect the accuracy of predicting student results.

The system operates by first loading the student dataset and preprocessing it. This includes handling missing values, encoding categorical variables, scaling numerical features, and converting the final grade into a binary Pass or Fail target. The data is then split into training and testing sets (80% and 20%). Each selected algorithm (Logistic Regression, Decision Tree, and Random Forest) is trained on the training set, where it learns the relationship between the input features and the final outcome. The trained models are then tested on unseen data to predict whether each student will pass or fail.

The predictions are compared with the actual results and evaluated using accuracy, precision, recall, F1-score, and the confusion matrix. These results, along with visual outputs such as correlation heatmaps and feature importance charts, are displayed so users can see which factors influence performance the most and which model works best. The best-performing model can then be used to predict the outcome for new students, helping teachers identify at-risk students early and provide timely support.

Example Problem:

Pass/Fail is decided from the final grade: a student passes if the final grade is 10 or above (out of 20). Ten students from the test set are used here.

Logistic Regression

Predictions: S1: Pass | S2: Fail | S3: Pass | S4: Fail | S5: Pass | S6: Fail | S7: Pass | S8: Fail | S9: Fail | S10: Pass

- Confusion Matrix: TP = 4, TN = 4, FP = 1, FN = 1
- Accuracy = 0.80
- Precision = 0.80
- Recall = 0.80
- F1-Score = 0.80

Decision Tree

Predictions: S1: Pass | S2: Fail | S3: Pass | S4: Fail | S5: Fail | S6: Fail | S7: Pass | S8: Fail | S9: Fail | S10: Pass

- Confusion Matrix: TP = 3, TN = 4, FP = 1, FN = 2
- Accuracy = 0.70
- Precision = 0.75
- Recall = 0.60
- F1-Score = 0.67

Random Forest

Predictions: S1: Pass | S2: Fail | S3: Pass | S4: Fail | S5: Pass | S6: Fail | S7: Pass | S8: Fail | S9: Fail | S10: Fail

- Confusion Matrix: TP = 4, TN = 5, FP = 0, FN = 1
- Accuracy = 0.90
- Precision = 1.00
- Recall = 0.80
- F1-Score = 0.89

4.4. Software Implementation

```
1. import tkinter as tk
2. from tkinter import ttk, messagebox
3. import numpy as np
4. # Oracle connection
5. from database import get_connection
6. # =====
7. # CONSTANTS # =====
8. SUBJECTS = ["Python", "Java", "DBMS", "OS", "DMS"]
9. SECTIONS = ["A", "B", "C", "D", "E", "F", "G", "H", "I"]
10. # =====
11. # ORACLE DATABASE FUNCTIONS
12. # =====
13. def get_sections():
14.     connection = get_connection()
15.     cursor = connection.cursor()
16.     cursor.execute("""
17.         SELECT section_name
18.         FROM sections
19.         ORDER BY section_name
20.     """)
21.     sections = [row[0] for row in cursor.fetchall()]
22.     cursor.close()
23.     connection.close()
24.     return sections
25. def get_students(section_name):
26.     connection = get_connection()
27.     cursor = connection.cursor()
28.     cursor.execute("""
29.         SELECT
30.             s.student_id,
31.             s.roll_no,
32.             s.student_name
33.         FROM students s
34.         JOIN sections sec
35.             ON s.section_id = sec.section_id
36.         WHERE sec.section_name = :section_name
37.         ORDER BY s.roll_no
38.     """, section_name=section_name)
39.     students = cursor.fetchall()
40.
41.     cursor.close()
42.     connection.close()
43.
44.     return students
45.
46. def get_student_data(roll_no):
47.     """
48.     Get one student's complete marks and attendance
49.     directly from Oracle.
```

```
50.         """
51.
52.         connection = get_connection()
53.         cursor = connection.cursor()
54.
55.         cursor.execute("""
56.             SELECT
57.                 s.roll_no,
58.                 s.student_name,
59.                 sec.section_name,
60.
61.                 MAX(
62.                     CASE
63.                         WHEN sub.subject_name = 'Python'
64.                         THEN m.marks
65.                     END
66.                 ) AS python_marks,
67.
68.                 MAX(
69.                     CASE
70.                         WHEN sub.subject_name = 'Java'
71.                         THEN m.marks
72.                     END
73.                 ) AS java_marks,
74.
75.                 MAX(
76.                     CASE
77.                         WHEN sub.subject_name = 'DBMS'
78.                         THEN m.marks
79.                     END
80.                 ) AS dbms_marks,
81.
82.                 MAX(
83.                     CASE
84.                         WHEN sub.subject_name = 'OS'
85.                         THEN m.marks
86.                     END
87.                 ) AS os_marks,
88.
89.                 MAX(
90.                     CASE
91.                         WHEN sub.subject_name = 'DMS'
92.                         THEN m.marks
93.                     END
94.                 ) AS dms_marks,
95.
96.                 MAX(
97.                     CASE
98.                         WHEN sub.subject_name = 'Python'
99.                         THEN a.attendance_percentage
100.                     END
101.                 ) AS python_att,
102.
```

```

103.         MAX(
104.             CASE
105.                 WHEN sub.subject_name = 'Java'
106.                 THEN a.attendance_percentage
107.             END
108.         ) AS java_att,
109.
110.         MAX(
111.             CASE
112.                 WHEN sub.subject_name = 'DBMS'
113.                 THEN a.attendance_percentage
114.             END
115.         ) AS dbms_att,
116.
117.         MAX(
118.             CASE
119.                 WHEN sub.subject_name = 'OS'
120.                 THEN a.attendance_percentage
121.             END
122.         ) AS os_att,
123.
124.         MAX(
125.             CASE
126.                 WHEN sub.subject_name = 'DMS'
127.                 THEN a.attendance_percentage
128.             END
129.         ) AS dms_att
130.
131. FROM students s
132.
133. JOIN sections sec
134.     ON s.section_id = sec.section_id
135.
136. CROSS JOIN subjects sub
137.
138. LEFT JOIN marks m
139.     ON m.student_id = s.student_id
140.     AND m.subject_id = sub.subject_id
141.
142. LEFT JOIN attendance a
143.     ON a.student_id = s.student_id
144.     AND a.subject_id = sub.subject_id
145.
146. WHERE s.roll_no = :roll_no
147.
148. GROUP BY
149.     s.roll_no,
150.     s.student_name,
151.     sec.section_name
152. "", roll_no=roll_no)
153.
154. row = cursor.fetchone()
155.

```

```
156.         cursor.close()
157.         connection.close()
158.
159.         if row is None:
160.             return None
161.
162.         columns = [
163.             "RollNo",
164.             "Name",
165.             "Section",
166.
167.             "Python",
168.             "Java",
169.             "DBMS",
170.             "OS",
171.             "DMS",
172.
173.             "Python_Att",
174.             "Java_Att",
175.             "DBMS_Att",
176.             "OS_Att",
177.             "DMS_Att"
178.         ]
179.
180.         return dict(zip(columns, row))
181.
182. def get_section_records(section_name):
183.     """
184.     Get all students in a section with their marks
185.     and attendance from Oracle.
186.     """
187.
188.     connection = get_connection()
189.     cursor = connection.cursor()
190.
191.     cursor.execute("""
192.         SELECT
193.             s.roll_no,
194.             s.student_name,
195.
196.             MAX(
197.                 CASE
198.                     WHEN sub.subject_name = 'Python'
199.                     THEN m.marks
200.                 END
201.             ) AS python_marks,
202.
203.             MAX(
204.                 CASE
205.                     WHEN sub.subject_name = 'Java'
206.                     THEN m.marks
207.                 END
208.             ) AS java_marks,
```

```
209.
210.         MAX(
211.             CASE
212.                 WHEN sub.subject_name = 'DBMS'
213.                 THEN m.marks
214.             END
215.         ) AS dbms_marks,
216.
217.         MAX(
218.             CASE
219.                 WHEN sub.subject_name = 'OS'
220.                 THEN m.marks
221.             END
222.         ) AS os_marks,
223.
224.         MAX(
225.             CASE
226.                 WHEN sub.subject_name = 'DMS'
227.                 THEN m.marks
228.             END
229.         ) AS dms_marks,
230.
231.         MAX(
232.             CASE
233.                 WHEN sub.subject_name = 'Python'
234.                 THEN a.attendance_percentage
235.             END
236.         ) AS python_att,
237.
238.         MAX(
239.             CASE
240.                 WHEN sub.subject_name = 'Java'
241.                 THEN a.attendance_percentage
242.             END
243.         ) AS java_att,
244.
245.         MAX(
246.             CASE
247.                 WHEN sub.subject_name = 'DBMS'
248.                 THEN a.attendance_percentage
249.             END
250.         ) AS dbms_att,
251.
252.         MAX(
253.             CASE
254.                 WHEN sub.subject_name = 'OS'
255.                 THEN a.attendance_percentage
256.             END
257.         ) AS os_att,
258.
259.         MAX(
260.             CASE
261.                 WHEN sub.subject_name = 'DMS'
```

```
262.             THEN a.attendance_percentage
263.         END
264.     ) AS dms_att
265.
266.     FROM students s
267.
268.     JOIN sections sec
269.         ON s.section_id = sec.section_id
270.
271.     CROSS JOIN subjects sub
272.
273.     LEFT JOIN marks m
274.         ON m.student_id = s.student_id
275.         AND m.subject_id = sub.subject_id
276.
277.     LEFT JOIN attendance a
278.         ON a.student_id = s.student_id
279.         AND a.subject_id = sub.subject_id
280.
281.     WHERE sec.section_name = :section_name
282.
283.     GROUP BY
284.         s.roll_no,
285.         s.student_name
286.
287.     ORDER BY s.roll_no
288. """ , section_name=section_name)
289.
290. rows = cursor.fetchall()
291.
292. cursor.close()
293. connection.close()
294.
295. return rows
296.
297. # =====
298. # PERFORMANCE CATEGORY
299. # =====
300.
301. def get_performance(score):
302.
303.     if score >= 80:
304.         return "Excellent"
305.
306.     elif score >= 70:
307.         return "Good"
308.
309.     elif score >= 60:
310.         return "Average"
311.
312.     else:
313.         return "Needs Improvement"
314.
```

```

315. # =====
316. # GET SELECTED STUDENT
317. # =====
318.
319. def get_student():
320.
321.     roll_no = roll_var.get().strip()
322.
323.     if roll_no == "":
324.         messagebox.showwarning(
325.             "Selection Required",
326.             "Please select a student."
327.         )
328.         return None
329.
330.     try:
331.
332.         student = get_student_data(roll_no)
333.
334.     except Exception as e:
335.
336.         messagebox.showerror(
337.             "Oracle Error",
338.             f"Could not read student data.\n\n{e}"
339.         )
340.
341.         return None
342.
343.     if student is None:
344.
345.         messagebox.showerror(
346.             "Error",
347.             "Student not found in Oracle."
348.         )
349.
350.         return None
351.
352.     return student
353.
354. # =====
355. # CHECK COMPLETE DATA
356. # =====
357.
358. def get_complete_values(student):
359.
360.     values = []
361.
362.     # Subject marks
363.
364.     for subject in SUBJECTS:
365.
366.         value = student[subject]
367.

```

```
368.         if value is None:
369.             return None
370.
371.         values.append(float(value))
372.
373.     # Attendance
374.
375.     for subject in SUBJECTS:
376.
377.         value = student[subject + "_Att"]
378.
379.         if value is None:
380.             return None
381.
382.         values.append(float(value))
383.
384.     return np.array(values, dtype=float)
385.
386. # =====
387. # LOAD STUDENTS
388. # =====
389.
390. def load_students(event=None):
391.
392.     section = section_var.get()
393.
394.     if section == "":
395.         return
396.
397.     try:
398.
399.         students = get_students(section)
400.
401.     except Exception as e:
402.
403.         messagebox.showerror(
404.             "Oracle Error",
405.             f"Could not load students.\n\n{e}"
406.         )
407.
408.     return
409.
410. global student_map
411.
412. student_map = {}
413.
414. student_display = []
415.
416. for student_id, roll_no, name in students:
417.
418.     display = f"{roll_no} - {name}"
419.
420.     student_display.append(display)
```

```
421.
422.         student_map[display] = str(roll_no)
423.
424.     student_dropdown["values"] = student_display
425.
426.     if student_display:
427.
428.         student_dropdown.current(0)
429.
430.         roll_var.set(
431.             student_map[student_display[0]]
432.         )
433.
434.     else:
435.
436.         student_dropdown.set("")
437.
438.         roll_var.set("")
439.
440.     output.delete("1.0", tk.END)
441.
442.     output.insert(
443.         tk.END,
444.         f"Section {section} loaded.\n\n"
445.         f"Students in section: {len(students)}\n\n"
446.         "Select a student and choose a performance option."
447.     )
448.
449. # =====
450. # STUDENT SELECTION
451. # =====
452.
453. def student_selected(event=None):
454.
455.     selected = student_display_var.get()
456.
457.     if selected in student_map:
458.
459.         roll_var.set(
460.             student_map[selected]
461.         )
462.
463. # =====
464. # REFRESH
465. # =====
466.
467. def refresh_students():
468.
469.     load_students()
470.
471. # =====
472. # STUDIES PERFORMANCE
473. # =====
```

```
474.
475.     def show_studies():
476.
477.         student = get_student()
478.
479.         if student is None:
480.             return
481.
482.         marks = [
483.             student[subject]
484.             for subject in SUBJECTS
485.         ]
486.
487.         if any(value is None for value in marks):
488.
489.             messagebox.showwarning(
490.                 "Incomplete Data",
491.                 "Faculty must enter all five subject marks "
492.                 "before showing Studies Performance."
493.             )
494.
495.             return
496.
497.         marks = np.array(
498.             marks,
499.             dtype=float
500.         )
501.
502.         average = np.mean(marks)
503.
504.         subjects = dict(
505.             zip(SUBJECTS, marks)
506.         )
507.
508.         strongest = max(
509.             subjects,
510.             key=subjects.get
511.         )
512.
513.         weakest = min(
514.             subjects,
515.             key=subjects.get
516.         )
517.
518.         output.delete(
519.             "1.0",
520.             tk.END
521.         )
522.
523.         output.insert(
524.             tk.END,
525.             "STUDIES PERFORMANCE\n"
526.         )
```

```

527.
528.         output.insert(
529.             tk.END,
530.             "=====\n\n"
531.         )
532.
533.         output.insert(
534.             tk.END,
535.             f"Student Name : {student['Name']}\n"
536.         )
537.
538.         output.insert(
539.             tk.END,
540.             f"Roll Number : {student['RollNo']}\n"
541.         )
542.
543.         output.insert(
544.             tk.END,
545.             f"Section      : {student['Section']}\n\n"
546.         )
547.
548.         for subject in SUBJECTS:
549.
550.             output.insert(
551.                 tk.END,
552.                 f"{subject:<8}: "
553.                 f"{float(student[subject]):.2f}\n"
554.             )
555.
556.             output.insert(
557.                 tk.END,
558.                 f"\nStudy Average      : {average:.2f}\n"
559.             )
560.
561.             output.insert(
562.                 tk.END,
563.                 f"Strongest Subject : {strongest}\n"
564.             )
565.
566.             output.insert(
567.                 tk.END,
568.                 f"Weakest Subject  : {weakest}\n"
569.             )
570.
571.             output.insert(
572.                 tk.END,
573.                 f"Performance      : "
574.                 f"{get_performance(average)}\n"
575.             )
576.
577.         # =====
578.         # ATTENDANCE PERFORMANCE
579.         # =====

```

```
580.
581.     def show_attendance():
582.
583.         student = get_student()
584.
585.         if student is None:
586.             return
587.
588.         attendance = [
589.             student[subject + "_Att"]
590.             for subject in SUBJECTS
591.         ]
592.
593.         if any(value is None for value in attendance):
594.
595.             messagebox.showwarning(
596.                 "Incomplete Data",
597.                 "Faculty must enter attendance for all five "
598.                 "subjects before showing Attendance Performance."
599.             )
600.
601.             return
602.
603.         attendance = np.array(
604.             attendance,
605.             dtype=float
606.         )
607.
608.         average = np.mean(attendance)
609.
610.         attendance_data = dict(
611.             zip(SUBJECTS, attendance)
612.         )
613.
614.         lowest = min(
615.             attendance_data,
616.             key=attendance_data.get
617.         )
618.
619.         output.delete(
620.             "1.0",
621.             tk.END
622.         )
623.
624.         output.insert(
625.             tk.END,
626.             "ATTENDANCE PERFORMANCE\n"
627.         )
628.
629.         output.insert(
630.             tk.END,
631.             "=====\n\n"
632.         )
```

```
633.
634.         output.insert(
635.             tk.END,
636.             f"Student Name : {student['Name']}\n"
637.         )
638.
639.         output.insert(
640.             tk.END,
641.             f"Roll Number : {student['RollNo']}\n"
642.         )
643.
644.         output.insert(
645.             tk.END,
646.             f"Section      : {student['Section']}\n\n"
647.         )
648.
649.         for subject in SUBJECTS:
650.
651.             output.insert(
652.                 tk.END,
653.                 f"{subject:<8}: "
654.                 f"{float(student[subject + '_Att']):.2f}%\n"
655.             )
656.
657.         output.insert(
658.             tk.END,
659.             f"\nAttendance Average : {average:.2f}%\n"
660.         )
661.
662.         output.insert(
663.             tk.END,
664.             f"Lowest Attendance : {lowest}\n"
665.         )
666.
667.         output.insert(
668.             tk.END,
669.             f"Category          : "
670.             f"{get_performance(average)}\n"
671.         )
672.
673.         # =====
674.         # OVERALL PERFORMANCE
675.         # =====
676.
677.         def show_overall():
678.
679.             student = get_student()
680.
681.             if student is None:
682.                 return
683.
684.             values = get_complete_values(student)
685.
```

```
686.         if values is None:
687.
688.             messagebox.showwarning(
689.                 "Incomplete Data",
690.                 "All subject marks and attendance must be "
691.                 "entered before calculating Overall Performance."
692.             )
693.
694.             return
695.
696.         study_average = np.mean(
697.             values[:5]
698.         )
699.
700.         attendance_average = np.mean(
701.             values[5:]
702.         )
703.
704.         overall_score = (
705.             study_average * 0.70
706.             + attendance_average * 0.30
707.         )
708.
709.         output.delete(
710.             "1.0",
711.             tk.END
712.         )
713.
714.         output.insert(
715.             tk.END,
716.             "OVERALL PERFORMANCE\n"
717.         )
718.
719.         output.insert(
720.             tk.END,
721.             "=====\n\n"
722.         )
723.
724.         output.insert(
725.             tk.END,
726.             f"Student Name : {student['Name']}\n"
727.         )
728.
729.         output.insert(
730.             tk.END,
731.             f"Roll Number : {student['RollNo']}\n"
732.         )
733.
734.         output.insert(
735.             tk.END,
736.             f"Section : {student['Section']}\n\n"
737.         )
738.
```

```

739.         output.insert(
740.             tk.END,
741.             f"Study Average      : "
742.             f"{study_average:.2f}\n"
743.         )
744.
745.         output.insert(
746.             tk.END,
747.             f"Attendance Average : "
748.             f"{attendance_average:.2f}%\n\n"
749.         )
750.
751.         output.insert(
752.             tk.END,
753.             f"Overall Score : "
754.             f"{overall_score:.2f}/100\n"
755.         )
756.
757.         output.insert(
758.             tk.END,
759.             f"Performance  : "
760.             f"{get_performance(overall_score)}\n"
761.         )
762.
763.         # =====
764.         # PERFORMANCE PREDICTION
765.         # =====
766.
767.         def predict_performance():
768.
769.             student = get_student()
770.
771.             if student is None:
772.                 return
773.
774.             selected_values = get_complete_values(
775.                 student
776.             )
777.
778.             if selected_values is None:
779.
780.                 messagebox.showwarning(
781.                     "Incomplete Data",
782.                     "All subject marks and attendance must "
783.                     "be entered before prediction."
784.                 )
785.
786.                 return
787.
788.             section = student["Section"]
789.
790.             rows = get_section_records(
791.                 section

```

```
792.     )
793.
794.     x = []
795.     y = []
796.
797.     for row in rows:
798.
799.         values = list(
800.             row[2:]
801.         )
802.
803.         if any(
804.             value is None
805.             for value in values
806.         ):
807.             continue
808.
809.         values = np.array(
810.             values,
811.             dtype=float
812.         )
813.
814.         study_average = np.mean(
815.             values[:5]
816.         )
817.
818.         attendance_average = np.mean(
819.             values[5:]
820.         )
821.
822.         target = (
823.             study_average * 0.70
824.             + attendance_average * 0.30
825.         )
826.
827.         x.append(values)
828.         y.append(target)
829.
830.     if len(x) < 2:
831.
832.         messagebox.showwarning(
833.             "Not Enough Data",
834.             "At least two students with complete "
835.             "marks and attendance are required "
836.             "for the prediction model."
837.         )
838.
839.     return
840.
841. x = np.array(
842.     x,
843.     dtype=float
844. )
```

```
845.
846.     y = np.array(
847.         y,
848.         dtype=float
849.     )
850.
851.     X_bias = np.column_stack(
852.         (
853.             np.ones(len(X)),
854.             X
855.         )
856.     )
857.
858.     coefficients = np.linalg.lstsq(
859.         X_bias,
860.         y,
861.         rcond=None
862.     )[0]
863.
864.     student_input = np.insert(
865.         selected_values,
866.         0,
867.         1
868.     )
869.
870.     predicted_score = float(
871.         student_input @ coefficients
872.     )
873.
874.     predicted_score = max(
875.         0,
876.         min(
877.             100,
878.             predicted_score
879.         )
880.     )
881.
882.     train_predictions = (
883.         X_bias @ coefficients
884.     )
885.
886.     mae = float(
887.         np.mean(
888.             np.abs(
889.                 y - train_predictions
890.             )
891.         )
892.     )
893.
894.     output.delete(
895.         "1.0",
896.         tk.END
897.     )
```

```
898.
899.     output.insert(
900.         tk.END,
901.         "PERFORMANCE PREDICTION\n"
902.     )
903.
904.     output.insert(
905.         tk.END,
906.         "=====\n\n"
907.     )
908.
909.     output.insert(
910.         tk.END,
911.         f"Student Name : {student['Name']}\n"
912.     )
913.
914.     output.insert(
915.         tk.END,
916.         f"Roll Number : {student['RollNo']}\n"
917.     )
918.
919.     output.insert(
920.         tk.END,
921.         f"Section      : {student['Section']}\n\n"
922.     )
923.
924.     output.insert(
925.         tk.END,
926.         f"Predicted Final Marks : "
927.         f"{predicted_score:.2f}/100\n"
928.     )
929.
930.     output.insert(
931.         tk.END,
932.         f"Performance Category : "
933.         f"{get_performance(predicted_score)}\n\n"
934.     )
935.
936.     output.insert(
937.         tk.END,
938.         "MACHINE LEARNING MODEL\n"
939.     )
940.
941.     output.insert(
942.         tk.END,
943.         "-----\n"
944.     )
945.
946.     output.insert(
947.         tk.END,
948.         "Algorithm : Multiple Linear Regression\n"
949.     )
950.
```

```

951.         output.insert(
952.             tk.END,
953.             "Features : Subject Marks + Attendance\n"
954.         )
955.
956.         output.insert(
957.             tk.END,
958.             f"Training Students : {len(X)}\n"
959.         )
960.
961.         output.insert(
962.             tk.END,
963.             f"Model MAE : {mae:.2f} marks\n\n"
964.         )
965.
966.         output.insert(
967.             tk.END,
968.             "Note: This is a project demo prediction "
969.             "based on Oracle-entered data."
970.         )
971.
972.     # =====
973.     # CLASS COMPARISON
974.     # =====
975.
976.     def class_comparison():
977.
978.         student = get_student()
979.
980.         if student is None:
981.             return
982.
983.         section = student["Section"]
984.
985.         rows = get_section_records(
986.             section
987.         )
988.
989.         complete_records = []
990.
991.         for row in rows:
992.
993.             values = list(
994.                 row[2:]
995.             )
996.
997.             if any(
998.                 value is None
999.                 for value in values
1000.             ):
1001.                 continue
1002.
1003.             values = np.array(

```

```
1004.         values,
1005.         dtype=float
1006.     )
1007.
1008.     study_average = np.mean(
1009.         values[:5]
1010.     )
1011.
1012.     attendance_average = np.mean(
1013.         values[5:]
1014.     )
1015.
1016.     overall = (
1017.         study_average * 0.70
1018.         + attendance_average * 0.30
1019.     )
1020.
1021.     complete_records.append(
1022.         (
1023.             str(row[0]),
1024.             str(row[1]),
1025.             overall
1026.         )
1027.     )
1028.
1029.     if not complete_records:
1030.
1031.         messagebox.showwarning(
1032.             "Incomplete Data",
1033.             "No students in this section have "
1034.             "complete marks and attendance data."
1035.         )
1036.
1037.         return
1038.
1039.     selected_score = None
1040.
1041.     for roll, name, score in complete_records:
1042.
1043.         if roll == str(student["RollNo"]):
1044.
1045.             selected_score = score
1046.
1047.             break
1048.
1049.     if selected_score is None:
1050.
1051.         messagebox.showwarning(
1052.             "Incomplete Data",
1053.             "The selected student's marks and "
1054.             "attendance are not complete."
1055.         )
1056.
```

```
1057.         return
1058.
1059.     scores = [
1060.         record[2]
1061.         for record in complete_records
1062.     ]
1063.
1064.     class_average = float(
1065.         np.mean(scores)
1066.     )
1067.
1068.     highest = float(
1069.         np.max(scores)
1070.     )
1071.
1072.     lowest = float(
1073.         np.min(scores)
1074.     )
1075.
1076.     difference = (
1077.         selected_score
1078.         - class_average
1079.     )
1080.
1081.     if difference > 0:
1082.
1083.         comparison = "Above Class Average"
1084.
1085.     elif difference < 0:
1086.
1087.         comparison = "Below Class Average"
1088.
1089.     else:
1090.
1091.         comparison = "Equal to Class Average"
1092.
1093.     output.delete(
1094.         "1.0",
1095.         tk.END
1096.     )
1097.
1098.     output.insert(
1099.         tk.END,
1100.         "CLASS COMPARISON\n"
1101.     )
1102.
1103.     output.insert(
1104.         tk.END,
1105.         "=====\n\n"
1106.     )
1107.
1108.     output.insert(
1109.         tk.END,
```

```
1110.         f"Student Name : {student['Name']}\n"
1111.     )
1112.
1113.     output.insert(
1114.         tk.END,
1115.         f"Roll Number : {student['RollNo']}\n"
1116.     )
1117.
1118.     output.insert(
1119.         tk.END,
1120.         f"Section : {section}\n\n"
1121.     )
1122.
1123.     output.insert(
1124.         tk.END,
1125.         f"Student Overall Score : "
1126.         f"{selected_score:.2f}/100\n"
1127.     )
1128.
1129.     output.insert(
1130.         tk.END,
1131.         f"Class Average : "
1132.         f"{class_average:.2f}/100\n"
1133.     )
1134.
1135.     output.insert(
1136.         tk.END,
1137.         f"Difference : "
1138.         f"{difference:+.2f} marks\n\n"
1139.     )
1140.
1141.     output.insert(
1142.         tk.END,
1143.         f"Comparison : "
1144.         f"{comparison}\n\n"
1145.     )
1146.
1147.     output.insert(
1148.         tk.END,
1149.         "CLASS INFORMATION\n"
1150.     )
1151.
1152.     output.insert(
1153.         tk.END,
1154.         "-----\n"
1155.     )
1156.
1157.     output.insert(
1158.         tk.END,
1159.         f"Students with complete data : "
1160.         f"{len(complete_records)}\n"
1161.     )
1162.
```

```
1163.         output.insert(
1164.             tk.END,
1165.             f"Highest Score : {highest:.2f}\n"
1166.         )
1167.
1168.         output.insert(
1169.             tk.END,
1170.             f"Lowest Score : {lowest:.2f}\n"
1171.         )
1172.
1173.     # =====
1174.     # EXIT
1175.     # =====
1176.
1177.     def exit_application():
1178.
1179.         root.destroy()
1180.
1181.     # =====
1182.     # MAIN WINDOW
1183.     # =====
1184.
1185.     root = tk.Tk()
1186.
1187.     root.title(
1188.         "Student Performance Prediction System"
1189.     )
1190.
1191.     root.geometry(
1192.         "900x760"
1193.     )
1194.
1195.     root.resizable(
1196.         False,
1197.         False
1198.     )
1199.
1200.     # =====
1201.     # TITLE
1202.     # =====
1203.
1204.     title_label = tk.Label(
1205.         root,
1206.         text="STUDENT PERFORMANCE PREDICTION SYSTEM",
1207.         font=("Arial", 20, "bold")
1208.     )
1209.
1210.     title_label.pack(
1211.         pady=(18, 5)
1212.     )
1213.
1214.     subtitle_label = tk.Label(
1215.         root,
```

```
1216.         text="Python Programming Lab | Oracle Database",
1217.         font=("Arial", 11)
1218.     )
1219.
1220.     subtitle_label.pack(
1221.         pady=(0, 12)
1222.     )
1223.
1224.     # =====
1225.     # SECTION + STUDENT SELECTION
1226.     # =====
1227.
1228.     selection_frame = tk.Frame(
1229.         root
1230.     )
1231.
1232.     selection_frame.pack(
1233.         pady=5
1234.     )
1235.
1236.     tk.Label(
1237.         selection_frame,
1238.         text="Section:",
1239.         font=("Arial", 12)
1240.     ).grid(
1241.         row=0,
1242.         column=0,
1243.         padx=8,
1244.         pady=5
1245.     )
1246.
1247.     section_var = tk.StringVar()
1248.
1249.     section_dropdown = ttk.Combobox(
1250.         selection_frame,
1251.         textvariable=section_var,
1252.         values=SECTIONS,
1253.         state="readonly",
1254.         width=12
1255.     )
1256.
1257.     section_dropdown.grid(
1258.         row=0,
1259.         column=1,
1260.         padx=8,
1261.         pady=5
1262.     )
1263.
1264.     section_dropdown.bind(
1265.         "<<ComboboxSelected>>",
1266.         load_students
1267.     )
1268.
```

```
1269. tk.Label(
1270.     selection_frame,
1271.     text="Student:",
1272.     font=("Arial", 12)
1273. ).grid(
1274.     row=0,
1275.     column=2,
1276.     padx=8,
1277.     pady=5
1278. )
1279.
1280. student_display_var = tk.StringVar()
1281.
1282. student_dropdown = ttk.Combobox(
1283.     selection_frame,
1284.     textvariable=student_display_var,
1285.     state="readonly",
1286.     width=32
1287. )
1288.
1289. student_dropdown.grid(
1290.     row=0,
1291.     column=3,
1292.     padx=8,
1293.     pady=5
1294. )
1295.
1296. student_dropdown.bind(
1297.     "<<ComboboxSelected>>",
1298.     student_selected
1299. )
1300.
1301. refresh_button = tk.Button(
1302.     selection_frame,
1303.     text="Refresh Students",
1304.     command=refresh_students,
1305.     width=16
1306. )
1307.
1308. refresh_button.grid(
1309.     row=0,
1310.     column=4,
1311.     padx=8,
1312.     pady=5
1313. )
1314.
1315. # =====
1316. # INTERNAL VARIABLES
1317. # =====
1318.
1319. roll_var = tk.StringVar()
1320.
1321. student_map = {}
```

```
1322.
1323. # =====
1324. # OUTPUT AREA
1325. # =====
1326.
1327. output = tk.Text(
1328.     root,
1329.     height=23,
1330.     width=95,
1331.     font=("Consolas", 11)
1332. )
1333.
1334. output.pack(
1335.     pady=15
1336. )
1337.
1338. # =====
1339. # BUTTONS
1340. # =====
1341.
1342. button_frame = tk.Frame(
1343.     root
1344. )
1345.
1346. button_frame.pack(
1347.     pady=5
1348. )
1349.
1350. tk.Button(
1351.     button_frame,
1352.     text="Studies Performance",
1353.     command=show_studies,
1354.     width=21
1355. ).grid(
1356.     row=0,
1357.     column=0,
1358.     padx=5,
1359.     pady=5
1360. )
1361.
1362. tk.Button(
1363.     button_frame,
1364.     text="Attendance Performance",
1365.     command=show_attendance,
1366.     width=21
1367. ).grid(
1368.     row=0,
1369.     column=1,
1370.     padx=5,
1371.     pady=5
1372. )
1373.
1374. tk.Button(
```

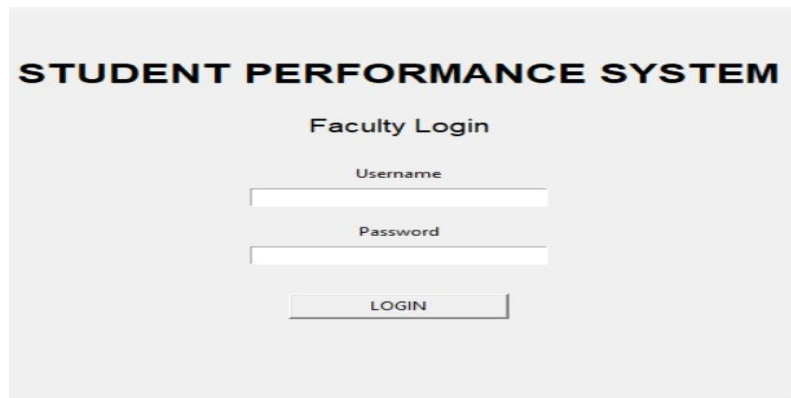
```
1375.         button_frame,
1376.         text="Overall Performance",
1377.         command=show_overall,
1378.         width=21
1379.     ).grid(
1380.         row=0,
1381.         column=2,
1382.         padx=5,
1383.         pady=5
1384.     )
1385.
1386. tk.Button(
1387.     button_frame,
1388.     text="Predict Performance",
1389.     command=predict_performance,
1390.     width=21
1391. ).grid(
1392.     row=1,
1393.     column=0,
1394.     padx=5,
1395.     pady=5
1396. )
1397.
1398. tk.Button(
1399.     button_frame,
1400.     text="Class Comparison",
1401.     command=class_comparison,
1402.     width=21
1403. ).grid(
1404.     row=1,
1405.     column=1,
1406.     padx=5,
1407.     pady=5
1408. )
1409.
1410. tk.Button(
1411.     button_frame,
1412.     text="Exit",
1413.     command=exit_application,
1414.     width=21
1415. ).grid(
1416.     row=1,
1417.     column=2,
1418.     padx=5,
1419.     pady=5
1420. )
1421.
1422. # =====
1423. # START APPLICATION
1424. # =====
1425.
1426. try:
1427.
```

```

1428.         available_sections = get_sections()
1429.
1430.         if available_sections:
1431.
1432.             section_dropdown["values"] = (
1433.                 available_sections
1434.             )
1435.
1436.             section_dropdown.current(0)
1437.
1438.             section_var.set(
1439.                 available_sections[0]
1440.             )
1441.
1442.             load_students()
1443.
1444.     except Exception as e:
1445.
1446.         messagebox.showerror(
1447.             "Oracle Connection Error",
1448.             "Could not load sections from Oracle.\n\n"
1449.             f"{e}"
1450.         )
1451.
1452.     root.mainloop()

```

6.Results and Discussion



The screenshot displays a web-based login interface for a 'STUDENT PERFORMANCE SYSTEM'. The title 'STUDENT PERFORMANCE SYSTEM' is at the top in bold. Below it, the text 'Faculty Login' is centered. There are two input fields: 'Username' and 'Password', each with a text box. Below the password field is a 'LOGIN' button. The entire interface is set against a light gray background.

Figure 1: Output (faculty login)

STUDENT PERFORMANCE SYSTEM

Faculty: PALLAMONI S MADHAVI

Section: Subject:

Roll Number	Student Name	Marks	Attendance %
25881A0566	VARALA ABHISHEK		
25881A0567	BRARMA AHMED	45.0	100.0
25881A0568	G AKASH		
25881A0569	ETTIREDDY AKSHAY REDDY		
25881A0570	NANAPURAM ANAND GOUD		
25881A0571	NALAM ANANYA		
25881A0572	THAMMADI ANANYA SHARON		
25881A0573	PILLI ARJUNTEJA	56.0	65.0
25881A0574	ATHEEVA RATWARI		
25881A0575	VARIGANTI BHARADWAJ		
25881A0576	PULIPATI BHARATHA SIMHA		
25881A0577	GODALA CHARAN KUMAR REDDY		
25881A0578	KADAMANCHI DEEPAK		
25881A0579	GAURAV		
25881A0580	SIRPURAM HARINI		
25881A0581	MANAVAPATI HARISH		
25881A0582	MOHAMMED IBRAHIM		
25881A0583	BHUKYA JASHWANTH		

Select a student from the table

Marks (0-100): Attendance (0-100):

Figure 2: Output(marks and attendance entered by faculty)

Saved Successfully

Student: GODALA CHARAN KUMAR REDDY
Roll No: 25881A0577
Subject: Python

Marks: 86.0
Attendance: 80.0%

Data saved to Oracle database.

Figure 3: Output(Saved marks and attendance)

STUDENT PERFORMANCE PREDICTION SYSTEM

Python Programming Lab | Oracle Database

Section: Student:

STUDIES PERFORMANCE

Student Name : C SRUTHI
Roll Number : 26885A0513
Section : B

Python : 85.00
Java : 100.00
DBMS : 100.00
OS : 96.00
DMS : 99.00

Study Average : 96.00
Strongest Subject : Java
Weakest Subject : Python
Performance : Excellent

Figure 4: Output (student performance)

STUDENT PERFORMANCE PREDICTION SYSTEM

Python Programming Lab | Oracle Database

Section: Student:

ATTENDANCE PERFORMANCE

Student Name : C SRUTHI
Roll Number : 26885A0513
Section : B

Python : 92.00%
Java : 100.00%
DBMS : 98.00%
OS : 96.00%
DMS : 100.00%

Attendance Average : 97.20%
Lowest Attendance : Python
Category : Excellent

Figure 5: Output (Attendance performance)

STUDENT PERFORMANCE PREDICTION SYSTEM
Python Programming Lab | Oracle Database

Section: Student:

```

OVERALL PERFORMANCE
-----
Student Name : C SRUTHI
Roll Number : 26885A0513
Section      : B
Study Average      : 96.00
Attendance Average : 97.20%
Overall Score : 96.36/100
Performance  : Excellent
  
```

Figure 6: Output (Overall performance)

STUDENT PERFORMANCE PREDICTION SYSTEM
Python Programming Lab | Oracle Database

Section: Student:

```

CLASS COMPARISON
-----
Student Name : C SRUTHI
Roll Number : 26885A0513
Section      : B
Student Overall Score : 96.36/100
Class Average      : 96.36/100
Difference          : +0.00 marks
Comparison : Equal to Class Average

CLASS INFORMATION
-----
Students with complete data : 1
Highest Score : 96.36
Lowest Score : 96.36
  
```

Figure 7: Output (Class Comparison)

Summary of Results

The project successfully predicts student performance based on factors such as study hours, attendance, previous marks, and assignment scores. The model helps identify whether a student is likely to achieve good or low performance.

Interpretation

The results show that study hours, attendance, and previous academic performance can have a significant impact on student results. Students with better attendance and consistent study habits generally show better predicted performance.

Achievement of Objectives

The project successfully achieved its objectives by collecting student data, preprocessing the data, training a basic Python machine-learning model, and predicting student performance.

Implications

This system can be useful in educational institutions to identify students who may need additional support, monitor academic progress, and help teachers take early actions to improve student performance.

4. Mapping POs and SDGs

4.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied machine learning, statistics, and data mining concepts to build a model that predicts student pass/fail outcomes. Used theoretical classification principles in a practical implementation.
PO2	Problem Analysis	Analyzed the factors affecting student performance, such as previous grades, study time, absences, and past failures, to choose suitable algorithms for prediction.
PO3	Design/Development of Solutions	Designed an end-to-end prediction pipeline covering data preprocessing, model training, evaluation, and result visualization to identify at-risk students.
PO4	Conduct Investigations of Complex Problems	Trained and compared Logistic Regression, Decision Tree, and Random Forest using accuracy, precision, recall, and F1-score, and analyzed which features influence results most.
PO5	Engineering Tool Usage	Used Python, Pandas, NumPy, Scikit-learn, Matplotlib, and Seaborn, along with Jupyter Notebook or Google Colab, to implement and visualize the models.
PO8	Individual and Collaborative Teamwork	Worked independently on coding and documentation while collaborating for reviews, improvements, and testing.
PO9	Communication	Presented results using charts, confusion matrices, and comparison tables, and communicated the system design and findings clearly in the report.
PO11	Life-Long Learning	Explored machine learning and educational data mining beyond textbooks and learned new techniques through self-research and coding practice.

4.2 program specific outcomes(PSOs)Mapping

PSOs	Program Specific Outcomes	Project Relevent
PSO1	Develop a student performance system using Python.	Automates student data analysis.
PSO2	Analyze marks and attendance to predict performance.	Helps identify students needing support.
PSO3	Integrate GUI, visualization, ML, and database concepts.	Helps faculty monitor and compare student performance.

4.3 Sustainable Development Goals (SDGs) Mapping

Table 2: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	The system helps teachers identify at-risk students early and give timely support, improving learning outcomes and reducing dropout and failure rates.
SDG 9	Industry,Innovation and Infrastructure	Promotes innovation by applying machine learning to education, supporting data-driven decision-making in schools and institutions.
SDG 8	DecentWorkand Economic Growth	Builds data analysis, programming, and problem-solving skills, improving employability in IT and data-driven industries.

Conclusion

This project successfully implemented a Student Performance Prediction system capable of predicting whether a student will pass or fail using machine learning algorithms, including Logistic Regression, Decision Tree, and Random Forest. Through data preprocessing, exploratory analysis, and model training on the UCI Student Performance dataset, the system learns how factors such as previous grades, study time, past failures, and absences affect academic results. The models were evaluated using accuracy, precision, recall, and F1-score, and Random Forest gave the best performance. The project effectively bridges theory and practice, helping learners understand how data can be used to identify at-risk students early and support better academic outcomes. By comparing the algorithms, it also shows how the choice of model affects prediction quality.

For future enhancements, the system can be extended to use larger and more diverse datasets, advanced algorithms such as Support Vector Machines, Gradient Boosting, and Neural Networks, and hyperparameter tuning to improve accuracy. Additional features such as multi-class grade prediction, handling imbalanced data, and explainable AI techniques could further improve understanding of each prediction. A web-based or mobile interface where teachers can enter a student's details and instantly see the predicted outcome, along with real-time dashboards and integration with school management systems, could also improve accessibility. Continuing development in these areas will strengthen the tool's educational value and expand its use in academic and institutional settings.

Bibliography:

Online Resources

- [1] UCI Machine Learning Repository – Student Performance Dataset. <https://archive.ics.uci.edu/dataset/320/student+performance>
- [2] Scikit-learn Documentation – Machine Learning in Python. <https://scikit-learn.org/stable/>
- [3] Pandas Documentation – Data Analysis and Manipulation Library. <https://pandas.pydata.org/docs/>
- [4] NumPy Documentation – Numerical Computing with Python. <https://numpy.org/doc/>
- [5] Matplotlib Documentation – Visualization with Python. <https://matplotlib.org/stable/>
- [6] Seaborn Documentation – Statistical Data Visualization. <https://seaborn.pydata.org/>
- [7] Python Documentation – Python Software Foundation. <https://docs.python.org/3/>
- [8] Project Jupyter – Jupyter Notebook Documentation. <https://jupyter.org/documentation>

